HACKEA UNA APP MÓVIL

Máquina BANKING DockerLabs

BIG school

EL LADO DEL MAL

OBJETIVO DE LA PRÁCTICA

Vamos a atacar banking, una máquina de DockerLabs centrada en una app bancaria con una API cuyo cliente Android filtra más información de la que debería. Hay dos caminos distintos para llegar a admin, y aquí se documentan los dos:

1. **Resolución 1:** analizar el APK de la app para encontrar un endpoint de cambio de contraseña sin protección y usarlo contra el usuario admin.

2. **Resolución 2:** localizar un `.git` expuesto en el servidor, extraer de él el secreto usado para firmar los JWT, y falsificar con ese secreto un token de admin.

ANTES DE EMPEZAR

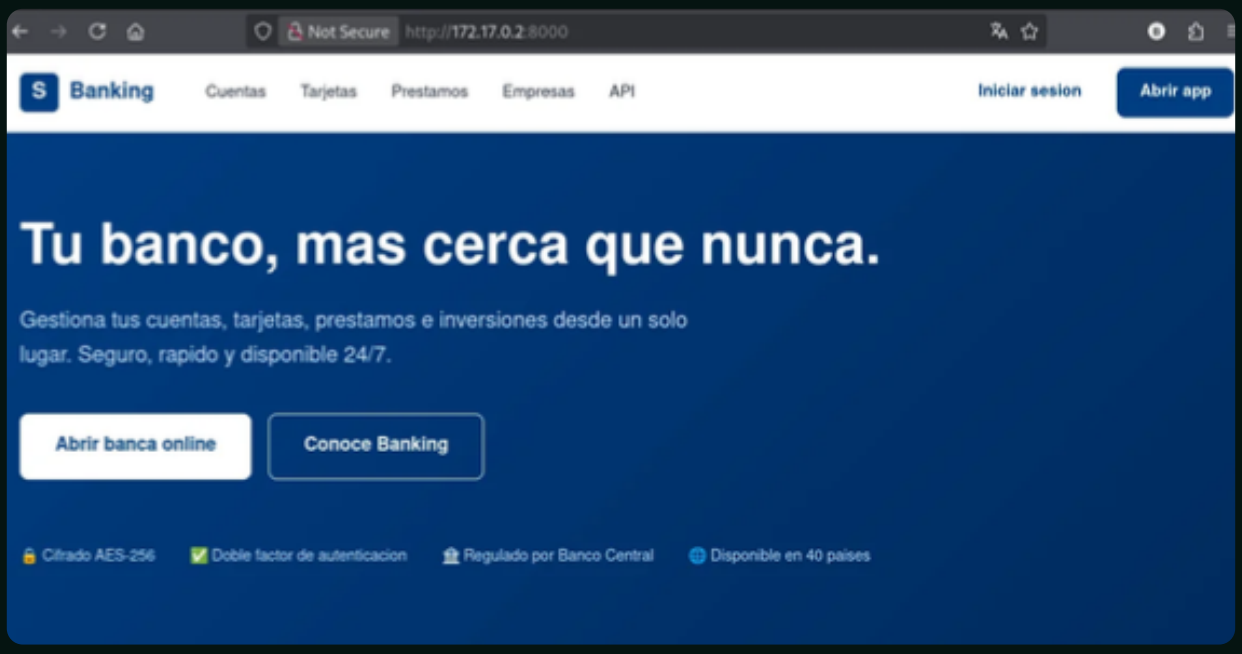
Todo lo que sigue se ejecuta contra una máquina de laboratorio (DockerLabs), pensada expresamente para practicar.
Las mismas técnicas contra un sistema real sin autorización son ilegales.

1) RECONOCIMIENTO

La tenemos disponible en bunkerlabs:



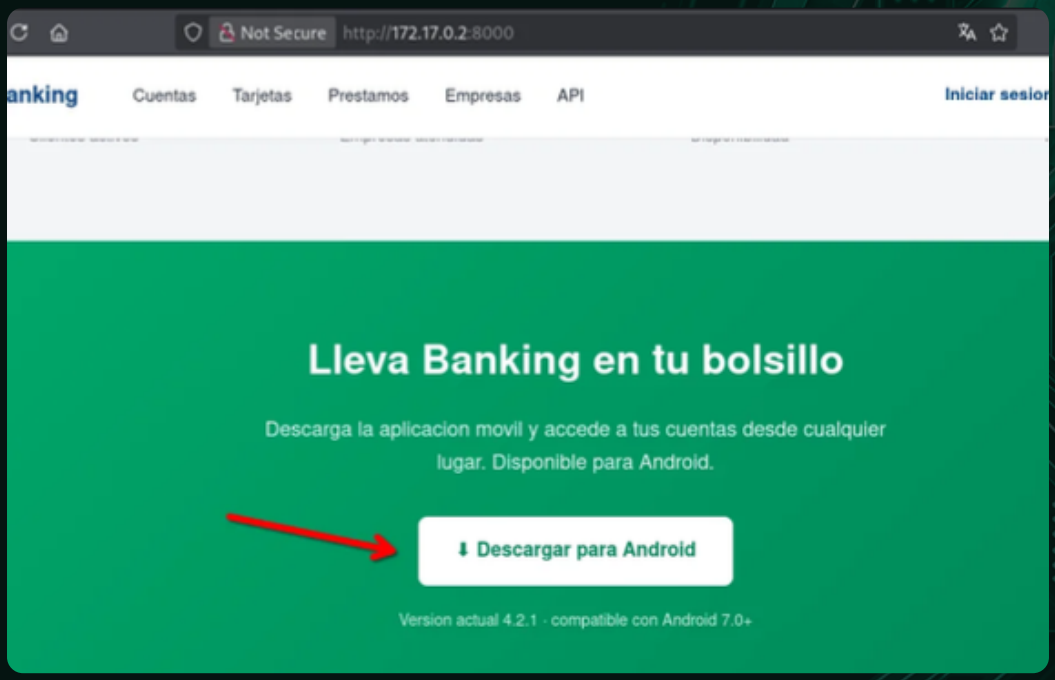
Una vez desplegada, vemos que por el puerto 8000 corre una web:



Resolución 1 — Endpoint de cambio de contraseña sin protección

1.1. Descargando y analizando el APK

En la parte de abajo de la web se ofrece la posibilidad de descargar un APK:



Nos lo bajamos y lo descomprimimos, donde encontramos lo siguiente:

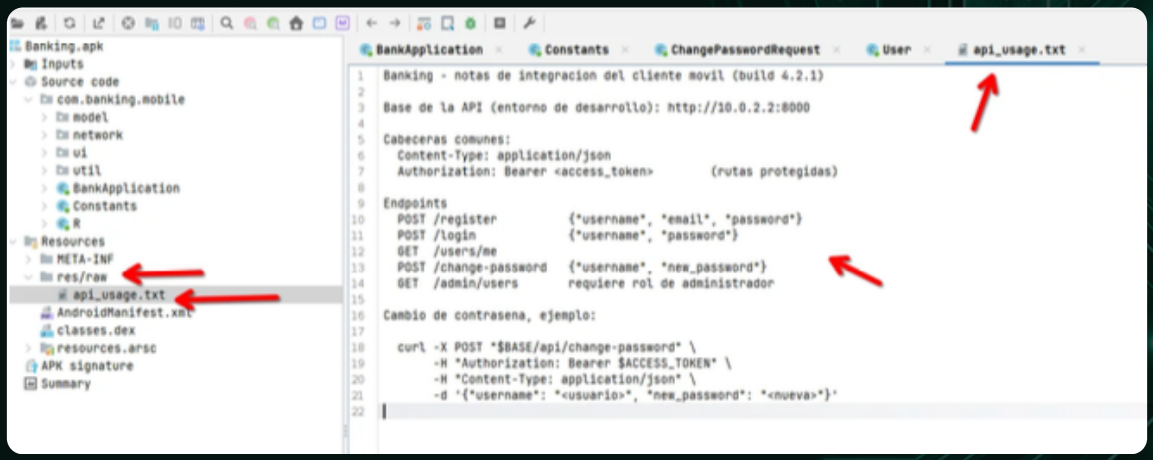
```
┌──(kali㉿kali)-[~/Desktop]
└─$ unzip Banking.apk
Archive:  Banking.apk
  inflating: AndroidManifest.xml
  inflating: META-INF/MANIFEST.MF
  inflating: META-INF/CERT.SF
  inflating: classes.dex
  inflating: lib/arm64-v8a/libflutter.so
  inflating: lib/x86_64/libflutter.so
  inflating: lib/armeabi-v7a/libflutter.so
  inflating: assets/flutter_assets/AssetManifest.json
  inflating: assets/flutter_assets/config/debug.env
  inflating: assets/flutter_assets/assets/secret_hint.txt
  inflating: assets/flutter_assets/FontManifest.json
  inflating: res/values/strings.xml
  inflating: res/raw/build_info.txt
  inflating: README.txt
  inflating: res/mipmap-mdpi/ic_launcher.png

┌──(kali㉿kali)-[~/Desktop]
└─$ ls
AndroidManifest.xml  assets  auto_deploy.sh  Banking.apk  banking.tar  classes.dex  lib  META-INF  README.txt  res
```

Aplicamos un filtro con `grep` buscando información delicada dentro de los ficheros del APK:

```
grep -r 'password' *
```

También podemos recurrir a **jadx** para decompilar la APK y revisar su código Java, donde se ve que el endpoint de cambio de contraseña no comprueba realmente de quién es la cuenta que se está modificando:

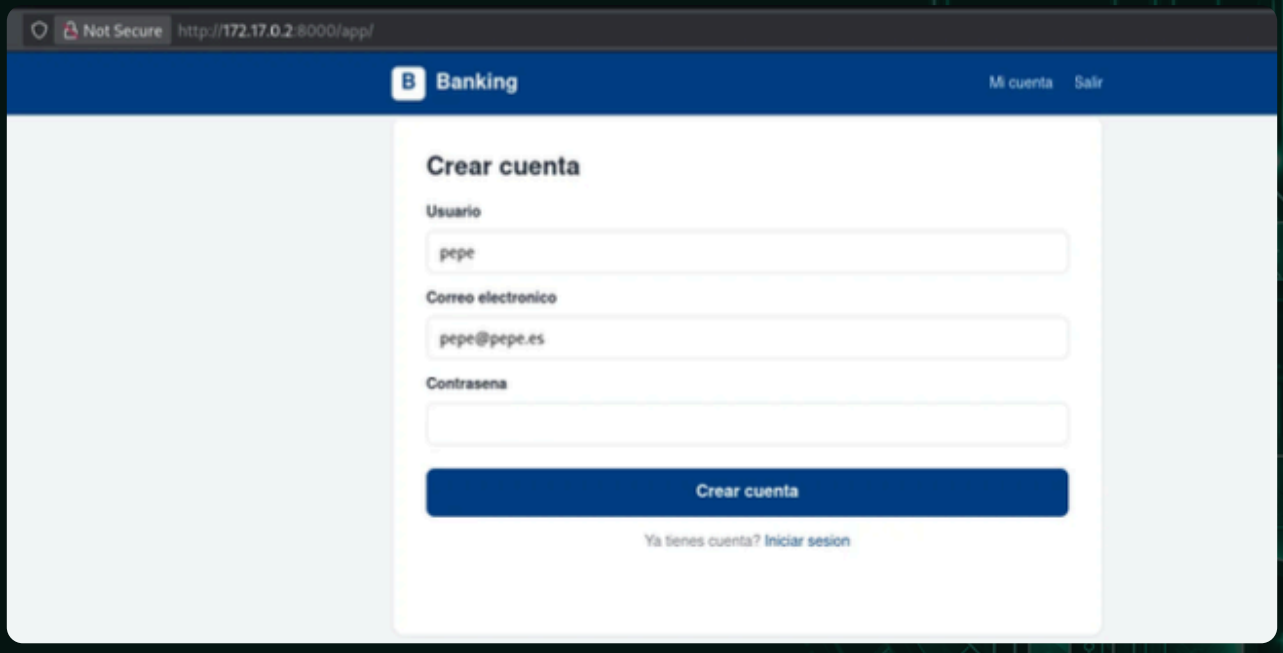


Vulnerabilidad - Cambio de contraseña sin control de propiedad (IDOR)

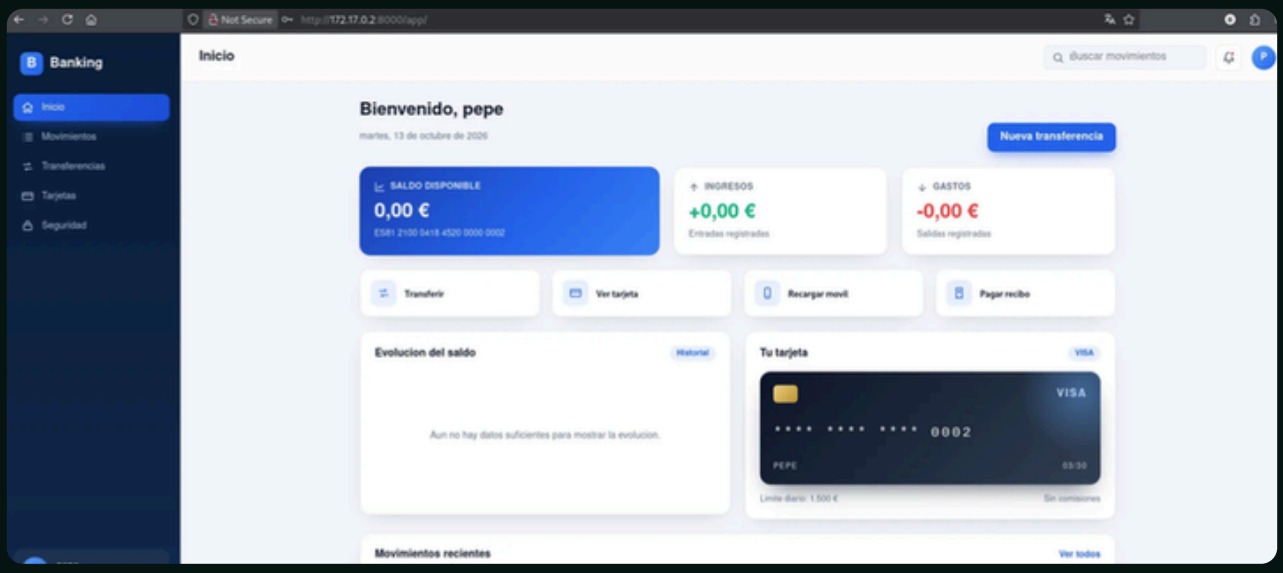
El endpoint de cambio de contraseña de la API recibe el usuario objetivo como parámetro y no verifica que coincida con el usuario autenticado en la petición. Esto permite que cualquier cuenta autenticada cambie la contraseña de **cualquier otro usuario**, incluido admin.

1.2. Explotación

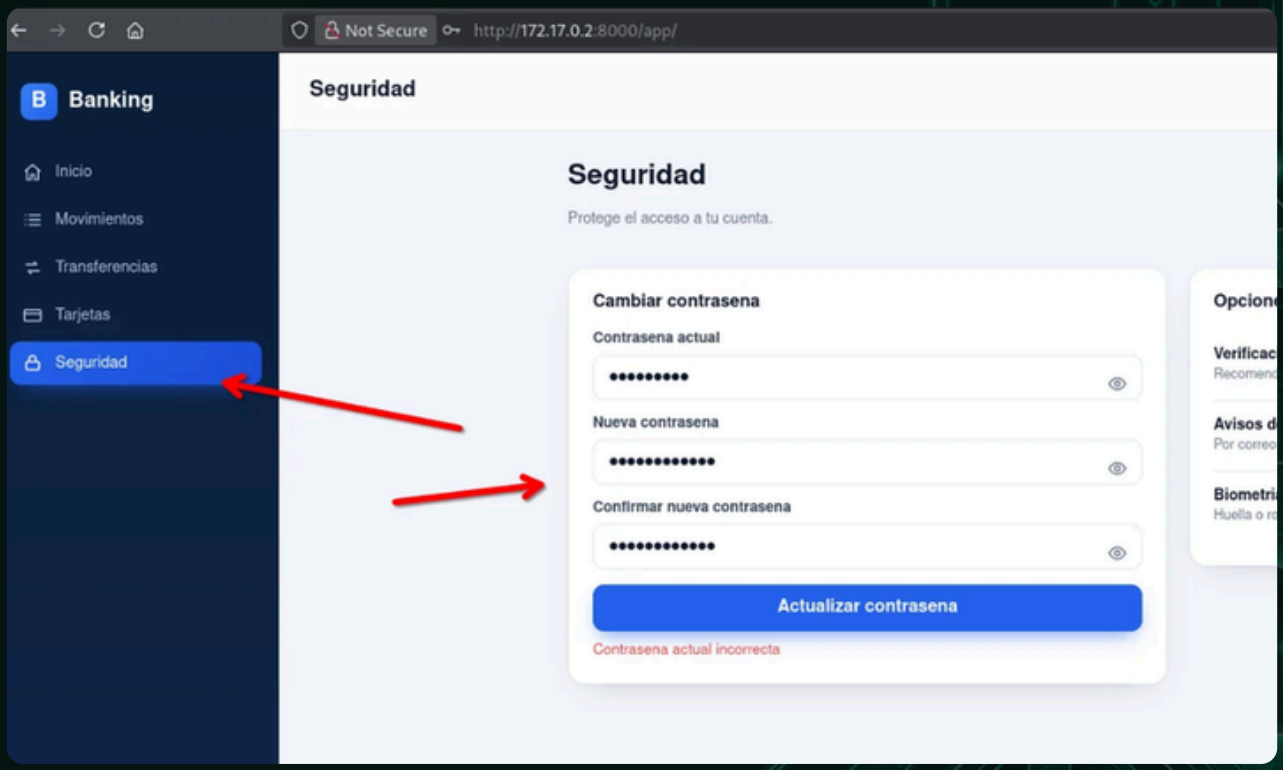
Con esta información ya sabemos el comando exacto para cambiar la contraseña de otro usuario, así que primero nos registramos en la app:



Y ya estamos dentro con nuestra propia cuenta:

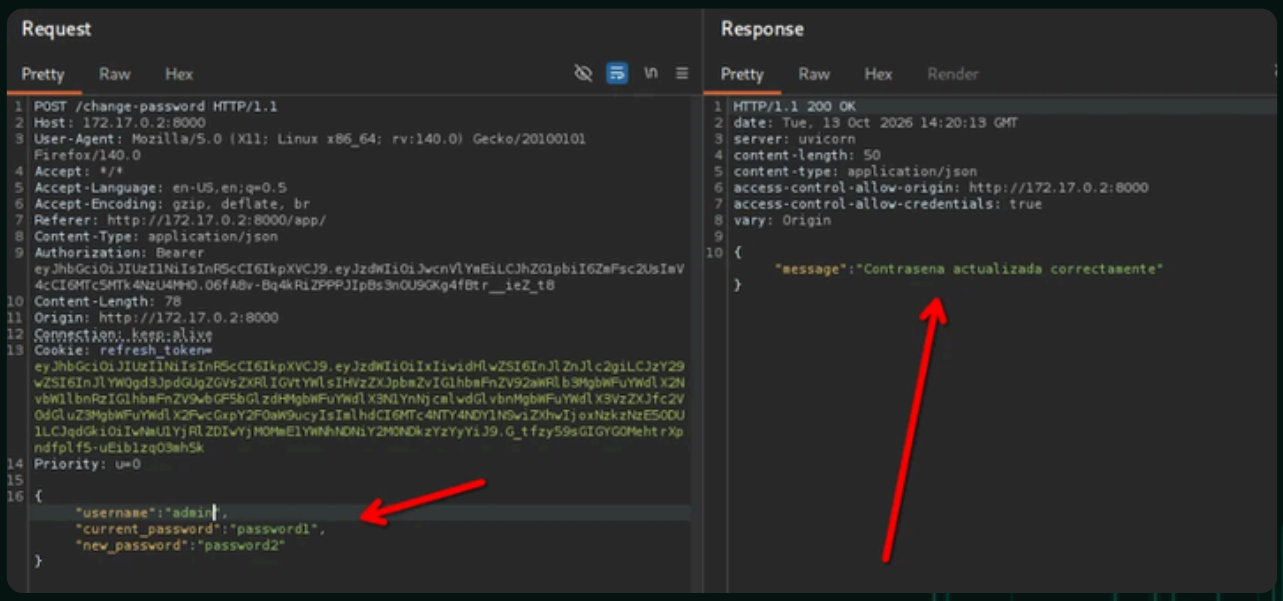


Desde dentro, revisamos la funcionalidad de cambio de contraseña:

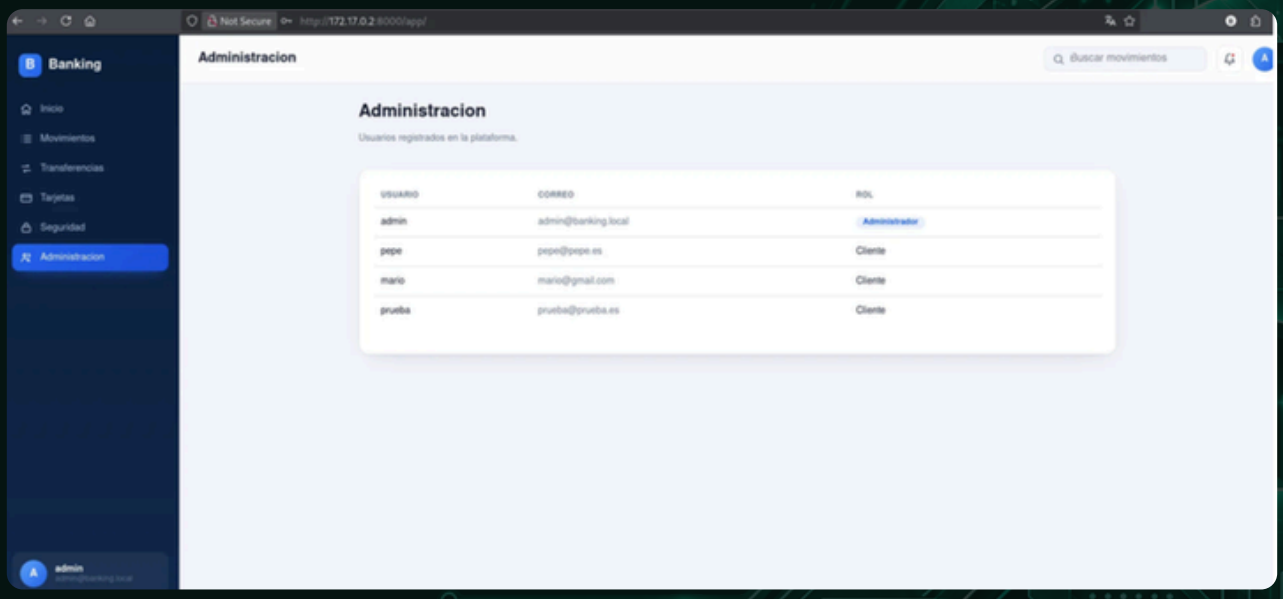


BIG school

Activamos la interceptación con Burp Suite en los ajustes del navegador, y cambiamos nuestra propia contraseña para ver cómo es la petición HTTP:



Si ahora iniciamos sesión como admin con la contraseña que le hemos cambiado, entramos sin problema:

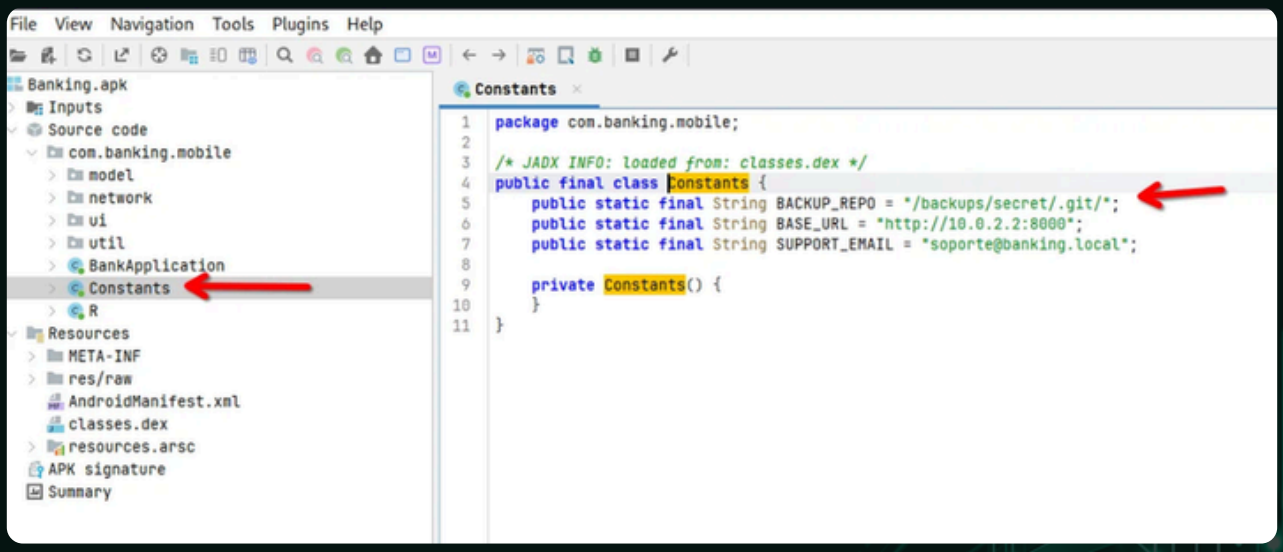


BIG school

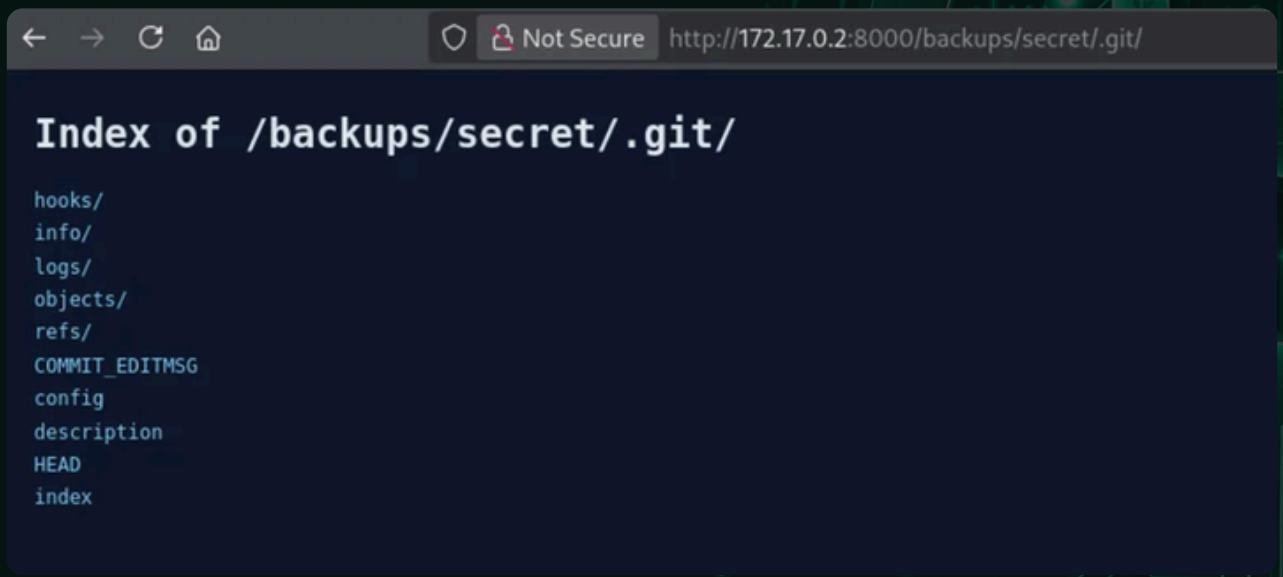
Resolución 2 — .git expuesto y falsificación del JWT

2.1. Descubriendo el repositorio expuesto

Si seguimos analizando el APK, vemos que en el servidor hay un directorio `.git` expuesto:



Al poner esa ruta en la URL, el servidor nos devuelve el contenido del `.git` mediante *directory listing*:



Vulnerabilidad - Repositorio .git expuesto en producción

El directorio .git del proyecto quedó accesible públicamente en el servidor. Esto permite descargar el historial completo del repositorio, incluyendo ficheros de configuración que no deberían estar expuestos (como el secreto usado para firmar los JWT).

2.2. Clonando el repositorio

Nos traemos todo el repositorio a nuestra máquina Kali Linux con `git clone` directamente contra esa ruta `.git`:

```
git clone http://172.17.0.2:8000/backups/secret/.git dump
```

```
┌──(kali㉿kali)-[~/Desktop]
└─$ git clone http://172.17.0.2:8000/backups/secret/.git dump

Cloning into 'dump' ...
```

Nos ubicamos en la carpeta clonada:

```
┌──(kali㉿kali)-[~/Desktop]
└─$ cd dump

┌──(kali㉿kali)-[~/Desktop/dump]
└─$ ls
auth.py  config.py  database.py  main.py  models.py  README.md  requirements.txt
```

2.3. Extrayendo el secreto JWT

Dentro del repositorio, revisamos config.py y encontramos el secreto usado para firmar los tokens JWT:

```
cat config.py
```

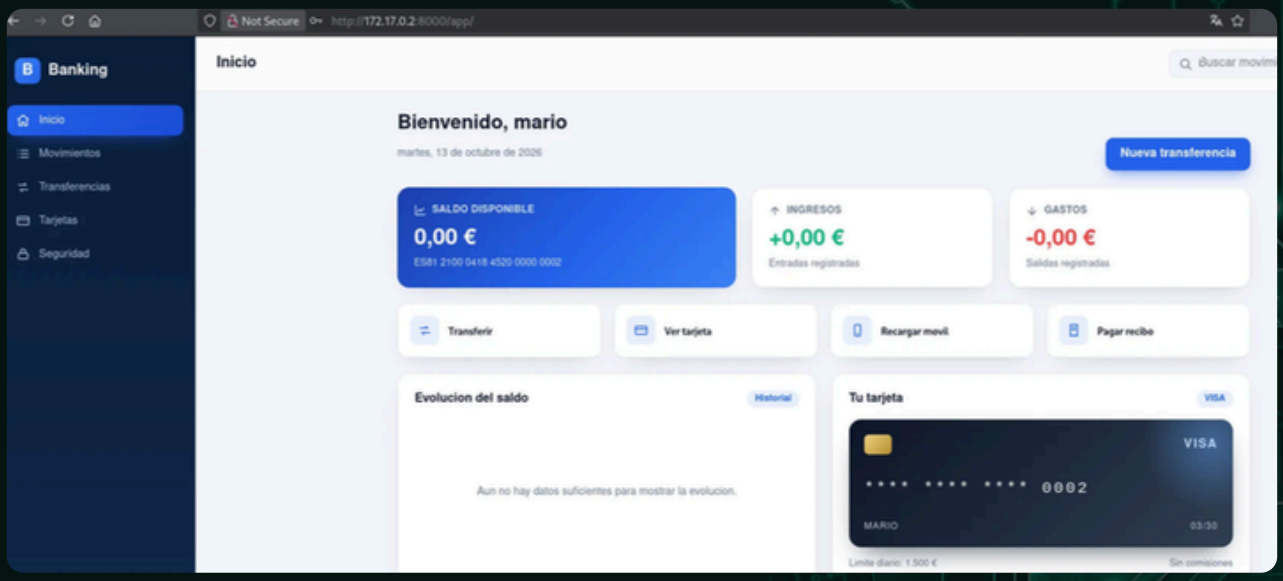
```
┌──(kali㉿kali)-[~/Desktop/dump]
└─$ cat config.py
import os

SECRET_KEY = os.getenv("SECRET_KEY", "supersecretkey-cambiar-en-produccion-12345")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 60 * 24

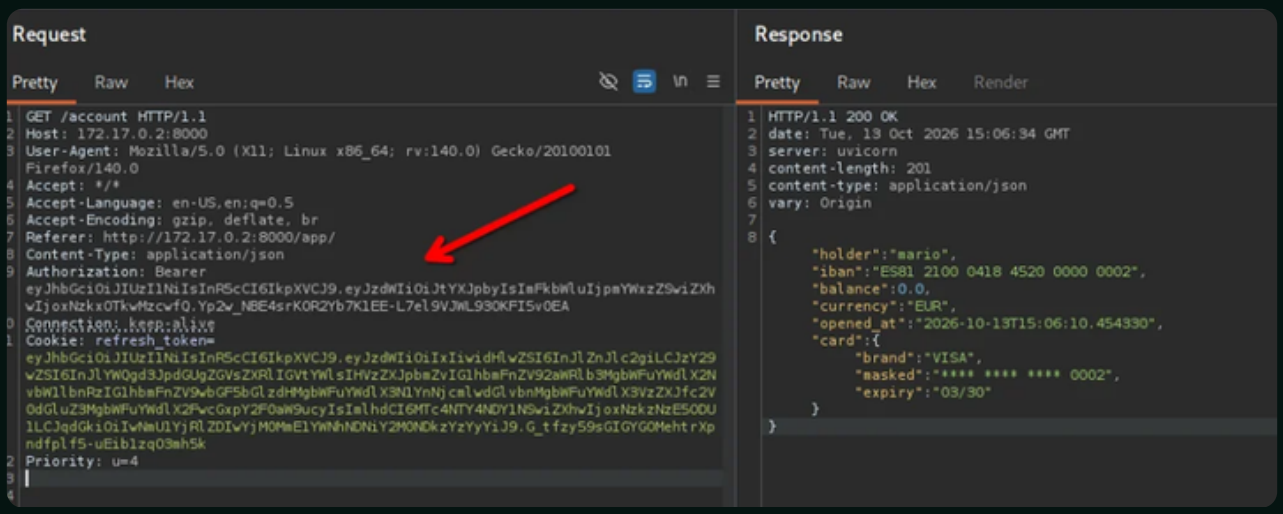
DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./banking.db")

INITIAL_ADMIN_USERNAME = "admin"
INITIAL_ADMIN_PASSWORD = os.getenv("INITIAL_ADMIN_PASSWORD", "")
```

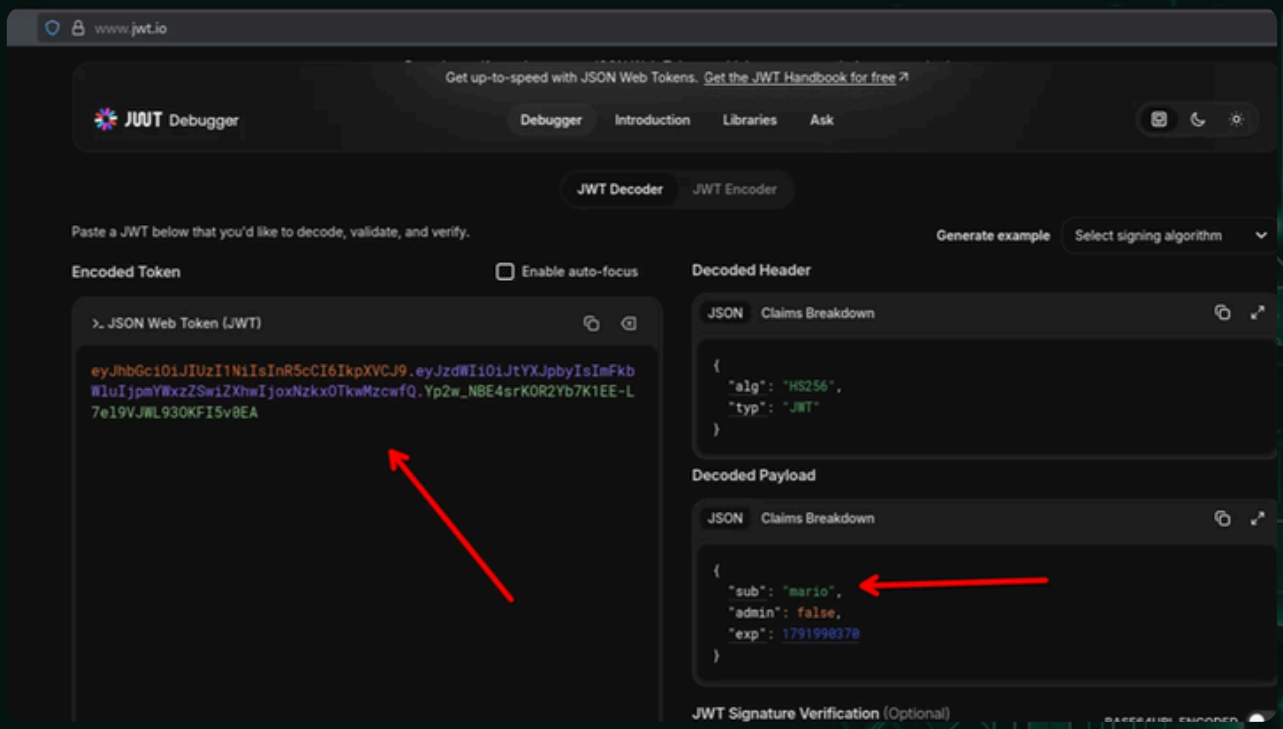
Con este secreto podemos iniciar sesión con nuestra propia cuenta de bajos privilegios:



Y en Burp Suite capturamos el token JWT asociado a nuestro usuario:



Si lo pegamos en jwt.io podemos ver el contenido (payload) del token:



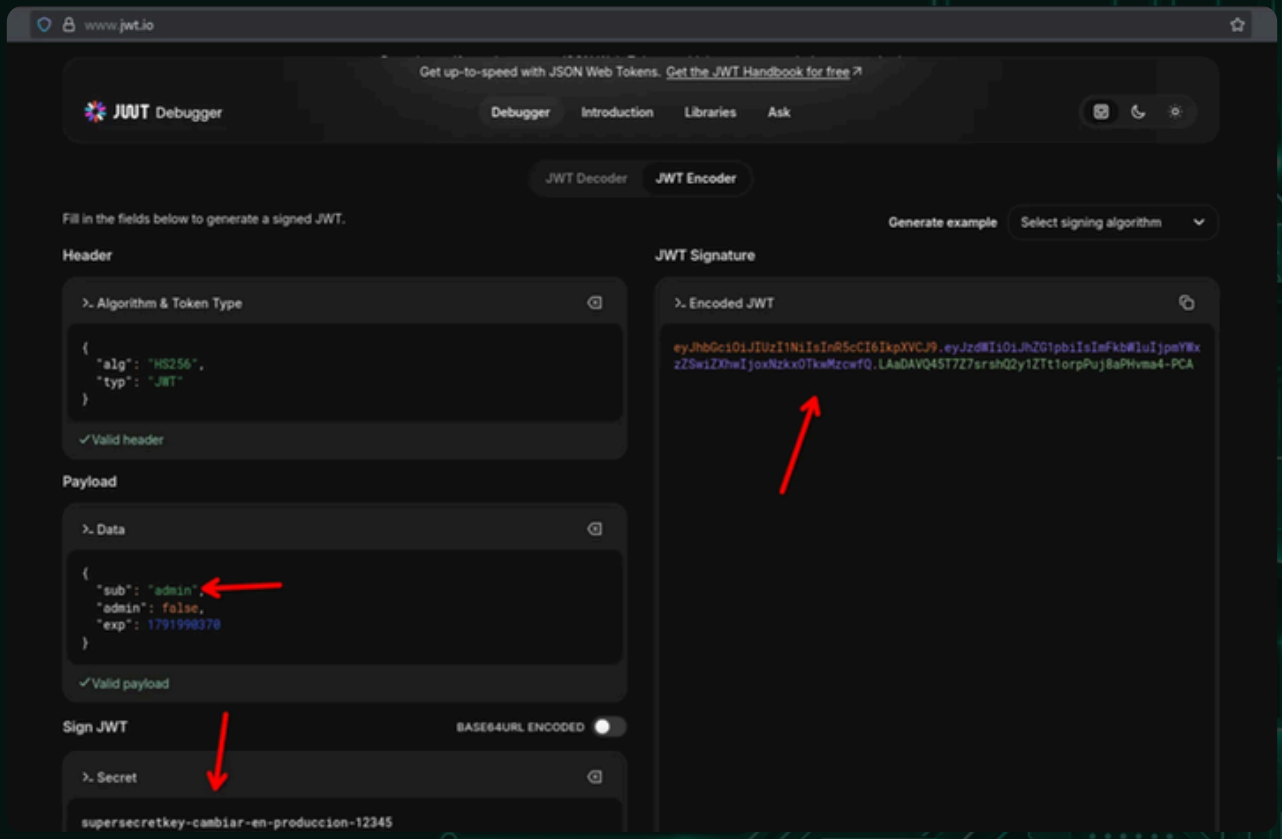
BIG school

¿Por qué importa tener el secreto de firma?

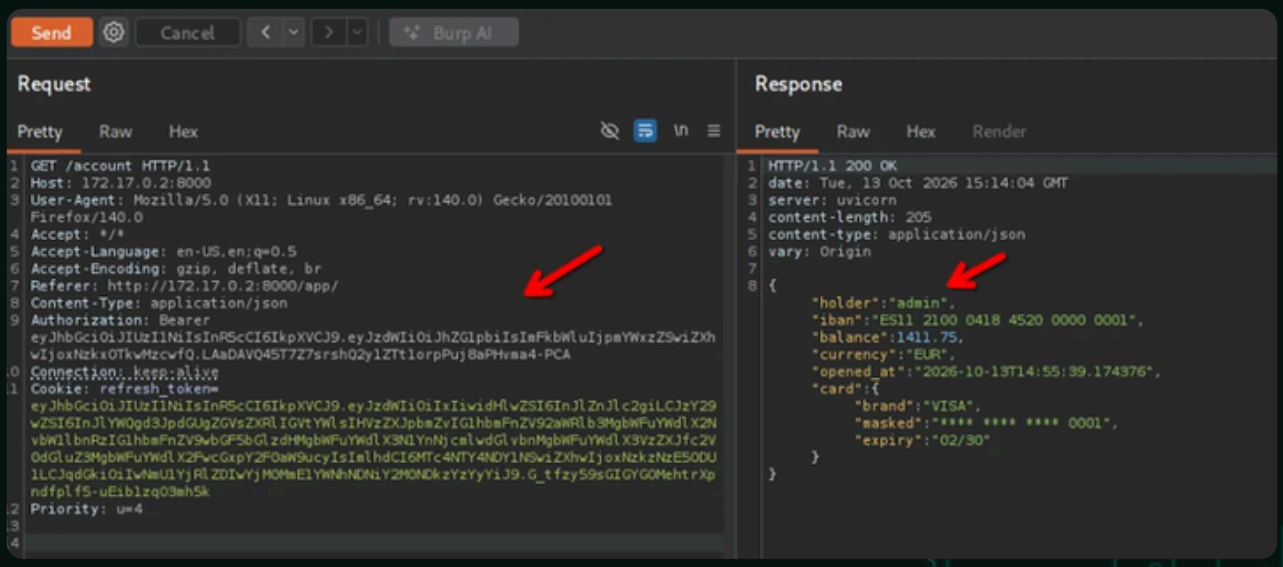
Un JWT va firmado con una clave secreta que sólo debería conocer el servidor. Si esa clave se filtra, cualquiera puede **generar tokens válidos a su gusto**, modificando el payload (por ejemplo, el campo de usuario o de rol) y volviendo a firmarlo con la misma clave — el servidor no tiene forma de distinguir ese token falsificado de uno legítimo.

2.4. Falsificando un token de admin

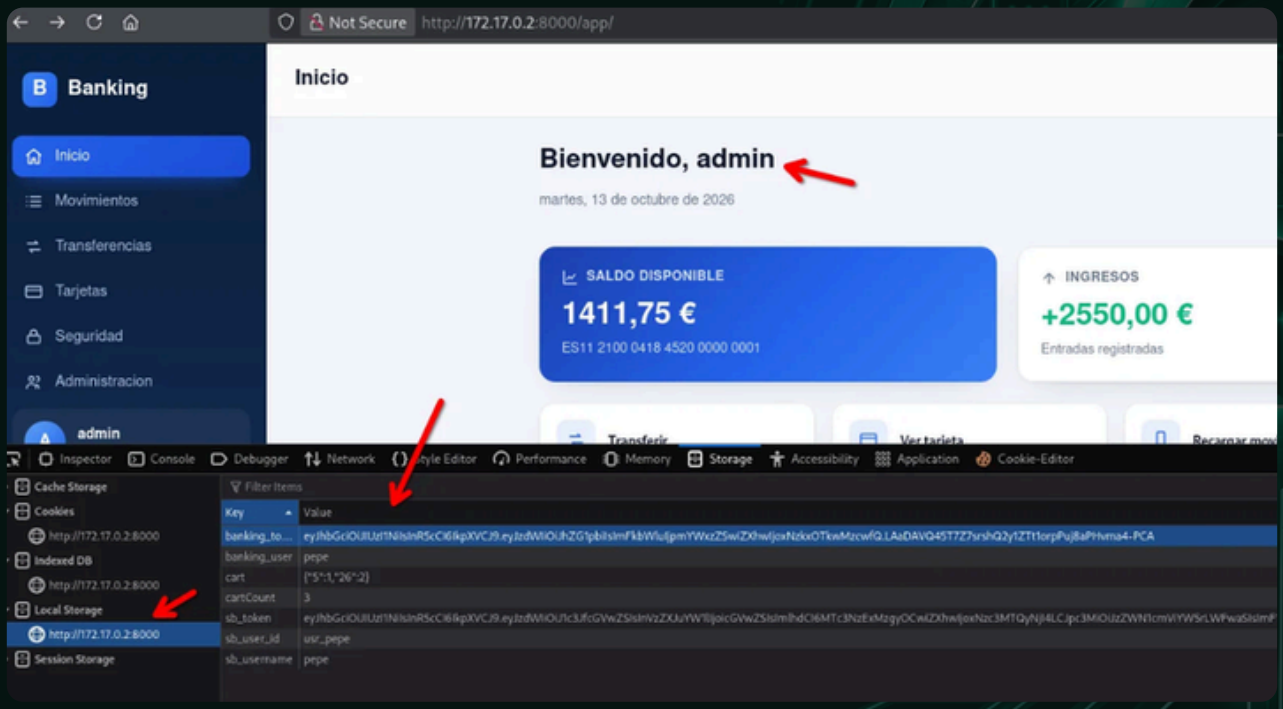
Como disponemos del secreto de firma, podemos modificar el payload del token y cambiar el usuario a admin, firmándolo de nuevo con la misma clave:



Pegamos este nuevo token en Burp Suite, sustituyendo el de nuestra petición, y comprobamos que ya actuamos como admin:



Por último, capturamos la petición que hace el propio navegador y le cambiamos el *bearer token* por el de admin, quedando autenticados como admin también desde la interfaz web:



BIG school

RESUMEN DE LA CADENA DE ATAQUE

Resolución 1 - IDOR en cambio de contraseña:

1. Descarga del APK publicado en la propia web de la app.

2. `grep` sobre los ficheros del APK en busca de información sensible → `api_usage.txt` instrucciones de la API.

3. Decompilación con jadx y localización del endpoint de cambio de contraseña, sin comprobación de propiedad sobre la cuenta objetivo.

4. Registro de una cuenta propia e interceptación con Burp Suite de la petición de cambio de contraseña.

5. Repetición de la petición cambiando el usuario objetivo a admin.

6. Login como admin con la contraseña recién asignada.

Resolución 2 - `.git` expuesto + JWT falsificado:

1. Descubrimiento de un directorio `.git` accesible públicamente en el servidor.

2. Clonado del repositorio completo con `git clone` contra la ruta `.git`.

3. Lectura de `config.py` y extracción del secreto de firma de los JWT.

4. Login con una cuenta propia y captura de su JWT en Burp Suite.

5. Edición del payload del token (usuario → admin) y refirmado con el secreto filtrado.

6. Sustitución del token en la petición (Burp y navegador) para operar como admin.



BIG school

BIG school





No te pierdas la Sesión 3

Activa la campanita para que YouTube te avise cuando empiece el directo

ACTIVA LA CAMPANITA